

JavaScript Function Expressions

[Back to JS page](#)

Table of Contents

- [JavaScript Function Expressions](#)
 - [What is a Function Expression?](#)
 - [Example 1](#)
 - [Anonymous Functions](#)
 - [Example 2](#)
 - [Functions Stored in Variables](#)
 - [Example 3](#)
 - [Function Declarations vs Function Expressions](#)
 - [Example 4](#)
 - [Hoisting](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

What is a Function Expression?

A JavaScript function can also be defined using an **expression**.

A function expression can be stored in a variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Function Expression?</h4>
<p id="demo"></p>

<script>
const x = function (a, b) { return a * b };
document.getElementById("demo").innerHTML = "Result: " + x(4, 3);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Anonymous Functions

A function expression can be stored in a variable without a function name, called an **anonymous function**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Anonymous Functions</h4>
<p id="demo"></p>

<script>
const myFunction = function() {
  return "Hello from anonymous function!";
};
document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Functions Stored in Variables

A function expression can be stored in a variable using `var` , `let` , or `const` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions Stored in Variables</h4>
<p id="demo"></p>

<script>
const square = function(x) {
  return x * x;
};
document.getElementById("demo").innerHTML = "Square of 7: " + square(7);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Function Declarations vs Function Expressions

A **function declaration** defines a named function. A **function expression** defines a function inside an expression:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Declaration vs Expression</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function Declaration
function sumDeclaration(a, b) {
    return a + b;
}

// Function Expression
const sumExpression = function(a, b) {
    return a + b;
};

document.getElementById("demo1").innerHTML = "Declaration: " + sumDeclaration(5, 3);
document.getElementById("demo2").innerHTML = "Expression: " + sumExpression(5, 3);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Hoisting

Function declarations are hoisted (can be called before they are defined). Function expressions are **NOT** hoisted:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Expressions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function declaration is hoisted – can be called before definition
document.getElementById("demo1").innerHTML = "Declaration hoisted: " + myDeclaration(5);

function myDeclaration(x) {
  return x * 2;
}

// Function expression is NOT hoisted – this would cause an error:
try {
  let result = myExpression(5);
  document.getElementById("demo2").innerHTML = "Expression: " + result;
} catch(err) {
  document.getElementById("demo2").innerHTML = "Expression error: " + err.message;
}

const myExpression = function(x) {
  return x * 2;
};
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Function Expressions](#)